Draft version June 1, 2026
Typeset using LaTeX **default** style in AASTeX7.0.1

Evolutionary Signatures in the Formation of Protostars:
A guide to Colin's code and the application of `radmc3d`

C. J. Bourque [1]

[1] *Middlebury College, Department of Physics and Astronomy*

1. INTRODUCTION

This code is a modern implementation of the work described in M. M. Dunham & E. I. Vorobyov (2012), which itself is the third publication in a series of papers describing synthetic observations of protostellar cores (C. H. Young & N. J. Evans 2005; M. M. Dunham et al. 2010). The goal of this work is to build a physical model describing the density profile surrounding the protostar (Section 2); to calculate the temperature profile and accompanying spectra (Section 3); and to interpret these output products (Section 4). The aforementioned physical model itself utilizes numerical solutions for collapsing protostellar cores which are described in S. Terebey et al. (1984) (hereafter TSC84). The base-line parameters used for generating a model-run are based on hydrodynamic simulations by E. I. Vorobyov & S. Basu (2005, 2006, 2010).

This document should serve as a guide to the `Python` scripts which have been written for building and simulating these models in 3 dimensions. These scripts can be found at https://github.com/colinbourque/star-formation. While some relevant physics will be mentioned in this document as necessary, readers are encouraged to consult the above cited works for further context and explanations of the physical motivations.

2. MAKING THE PHYSICAL MODELS

A "run" or "model-run" of the 3D code describes the set of density profiles and control files for a given model which are later fed in to `radmc3d`. Those components which are unique for a given timestep (`amr_grid.inp`, `density_profile.inp`, and `stars.inp`) are described in this section, and the "auxiliary" files—which are repeated for a given run—are saved for Section 3.

The code is currently written such that one singular script, `problem_setup.py` (or notebook: `problem_setup.ipynb`) is capable of generating all of these input and control files. This scripts leverages functions contained by `radmc_utils.py` for many of the steps in generating a model. A general schematic of the process followed by the `problem_setup.py` script is shown in Figure 1.

The `problem_setup.py` scripts sends all output files along absolute paths, meaning it can be run from any location so long as the variable `simdir` is set correctly, and the script has access to the auxillary files from the 2D code.

2.1. *Inputs to generate the model*

The most important input considered by the `problem_setup.py` script is a model parameters table describing the physical configuration of the system. In addition to this, however, a number of controls describing how the files for a run of models will be generated are hardcoded into the `problem_setup.py` script for simplicity. In general, these fall into three categories:

1. Directory linkages,
2. Parameters held constant, and
3. Control definitions and formatting for `radmc3d`.

The model parameters file contains all of the physical information which is taken from the hydrodynamic simulations of Vorobyov & Basu. As it stands, `problem_setup.py` looks for this table in a `/model_parameters/`

Email: cbourque@middlebury.edu, cbourque@wayne.edu, colinbourque327@gmail.com

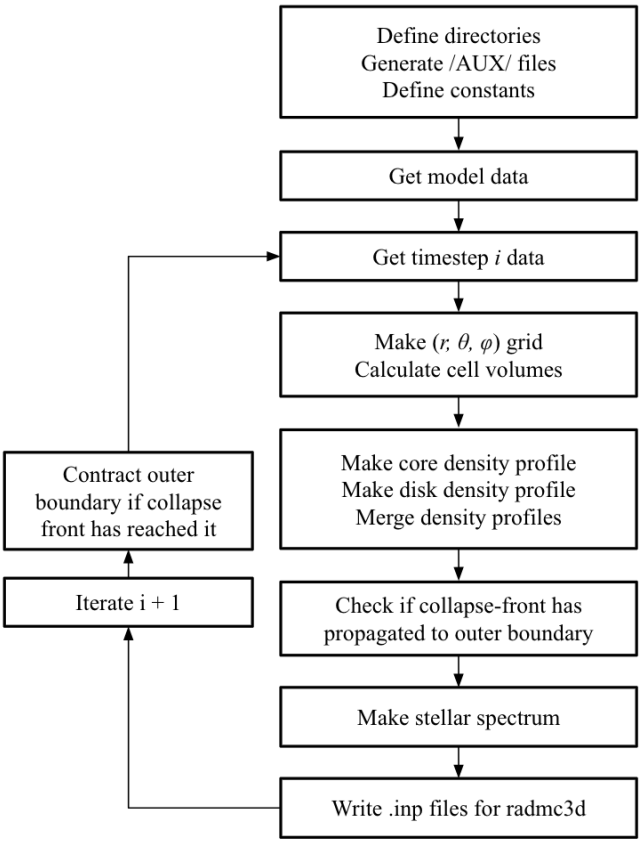

Figure 1. Schematic diagram showing the steps taken by the `problem_setup.py` script. These steps are explained in Section 2. This script takes a model definition as input, and returns control and input files used for running radiative transfer and raytracing calculations.

subdirectory for a file named `model_parameters_`n`.tbl`, where n specifies the model number. It is expected that this model parameters table specifies columns in a particular order and with particular units, which matches for example those included in the repository on GitHub. It is also expected that there may be rows that do not contain necessary physical data (e.g. the header, the $\Delta t = 0$ time step, and timesteps with `NaN` entries). These are handled in a few ways:

1. the header is skipped on import,
2. the first data entry is skipped by only beginning the model generation on the second timestep (index $i = 1$), and
3. if any row contains an `NaN` entry, it is dropped from the table.

Directory pointings are coded to be absolute (so `problem_setup.py` can be run from anywhere) but are written relative to a singular `simdir` location which will hold all of the model input, output, and analysis files (in various subdirectories). `simdir` is the first variable defined following physical constants, and can be changed to point to any location, so long as it is set up to contain a `model_parameters` table and the necessary subdirectories for writing the `radmc3d` input files (for an explanation of these directories, see Appendix A).

A number of **parameters are hold constant** in the code in order to match the physical model described in M. M. Dunham & E. I. Vorobyov (2012), although could theoretically be modified by changing the `problem_setup.py` file. These include factors such as *the number of points on the wavelength grid*, *the location of the central star*, or *the size of the higher-resolution θ grid region*. These are defined where necessary.

Finally, running `radmc3d` requires electing a number of decisions for controlling the radiative transfer code. These are coded directly into each block of code that generates a `radmc3d` control file, and the details of these decisions are explained in Section 3.1.

2.2. *Output model components*

In addition to the `radmc3d` control files, which are described in Section 3.1, the `problem_setup.py` script generates a set of grid, density, and stellar source definition files representing every (real) timestep in a model run. These are labeled using `radmc3d`'s naming convention with the addition of a three digit timestep identifier between the file name and extension.

These outputs files are given their respective formats as required by `radmc3d` (described in the documentation [2]). Example notebooks and scripts for reading and plotting `radmc3d` formatted files are included in the GitHub repository.

2.3. *Making each component of the model*

The following subsections will provide more context describing how each component of the model is built.

2.3.1. *The r, θ, ϕ grid*

The physical grid provided to `radmc3d` describes the boundaries of each cell in the simulation, and is expected to be 3-dimensional. If one dimension is not used (e.g., ϕ in the 2-D equivalent model) there is a control parameter to set in `radmc3d`'s inputs, and the grid then only needs two data points in this dimension such that it can span the full 3D domain. While `radmc3d` works on cell boundaries, computation of the physical model is done for singular points within these cells, and so requires a second grid of values describing these midpoints.

In `problem_setup.py` , the physical simulation space is described by the `model_paramaters` table, while the computed representation of the grid is controlled by electing the

1. number of radial points, `nr`
2. number of polar angle points, `nt`
3. number of azimuthal points, `nphi`
4. what angular range (in radians away from the midplane) the high-resolution grid will span, `tmps`, and
5. how many of the polar points will be left for outside of this refined region, `ntex`.

These parameters are then used to generate two grids of values

1. the radial midpoints grid, `rr`, with shape (nr, nt),
2. the theta midpoints grid, `tt`, with shape (nr, nt),

used for computing the model density. The boundaries lists, of shape $(n + 1)$, are stored for later use in calculating the volumes, as well as for writing to the `amr_grid.inp` file. At the moment, the azimuthal angle is hardcoded to only represent one cell spanning $[0, 2\pi]$. Since `numpy`'s `meshgrid` works for 3 dimensions, azimuthal dependence could be added to the model by following a similar procedure as θ for ϕ, as well.

The above grids are made through a call to the functions `make_r_grid()` and `make_theta_grid()`, which are packaged in the `radmc_utils` script. Re-defining the innermost radial cell center to match the innermost cell boundary is included in this call to make the radial grid. For all other points, the geometric mean is used to find midpoints in the r coordinate, and an arithmetic mean is used for the θ coordinate.

2.3.2. *The volumes grid*

A grid of volumes for each cell is generated by integrating over r, θ, ϕ for each cell's boundaries. In particular, two functions are defined in the `radmc_utils.py` script, which solve the r and θ components of this integral separately. There functions are called `r_integ()` and `theta_integ()`, and take the lower and upper bounds of each component of the integral as their arguments. In `problem_setup.py` these integral components are turned in to 2D grids of shape (nr, nt) and subsequently multiplied by each other along with a factor of 2π.

The result of the above calculation is a grid `cv` containing the volume of each cell, which is later used for computing total disk and core masses and subsequent normalization.

[2] https://www.ita.uni-heidelberg.de/~dullemond/software/radmc-3d/manual_radmc3d/index.html

2.3.3. *The accretion disk*

If the model parameters file specifies an accretion disk mass > 0, one is generated by solving Equation 11 of M. M. Dunham & E. I. Vorobyov (2012) over the above grid. The resulting density profile is then truncated to cut off at $s =$ `rdisk_out`. This disk is then normalized (using the cell volumes `cv` grid) to match to the total disk mass specified in the parameters file. If instead a disk mass of exactly 0 is specified, the above calculations are skipped, and an array of zeros with shape (nr, nt) is instead returned for the disk density profile.

2.3.4. *The core density profile*

A core density profile is generated by calling the function `make_cdp()` from `radmc_utils`. This function expects the r and θ grids as arguments, along with the initial rotation speed Ω_0, the model time, "the relative distortion of the spherical expansion wavefront" Δ_Q (TSC84), and the sound speed c_s. With these arguments set, a core density profile is generated following the procedure of TSC84, and a tuple with five values is returned:

1. the gas density at all simulation points with arbitrary normalization, of shape (nr, nt),
2. the radial velocity at all simulation points in units of cm s^{-1}, of shape (nr, nt),
3. the stretched similarity variable at all points (see TSC84 Fig. 3), of shape (nr, nt),
4. the similarity variable at all points (see TSC84 Fig. 2), of shape (nr, nt), and
5. the total rotation parameter τ (see TSC84 Eqn. 14), of shape () (just a float).

Similar to the disk density profile, the core density profile also gets truncated to only span `renv_in` $< r <$ `renv_out` and then normalized to match the model parameters file. The radial velocity at the outer boundary along the midplane is also checked and if it exceeds 0, the collapse velocity parameter is updated so the outer radial boundary for the subsequent timestep can be contracted accordingly.

For the function `make_cdp()` to work, it is necessary for the TSC84 numerically-solved parameters (`grid.plt34.mod`) to be stored in the same directory as `radmc_utils.py` and `problem_setup.py` (this is how the repository is packaged on GitHub already).

Generating the core density profile with `make_cdp()` begins with reading in TSC84's numerical values for y, α_0, α_M, α_Q, V_0, V_M, and V_Q. Grids representing the stretched similarity variables x and y are made, as well as a grid with computed values of $P_2(\cos\theta)$. Grids of each numerical parameter are then created by interpolation and extrapolation from the TSC84 data file using either `numpy`'s linear interpolation function or `extrap()`. Cascading if statements are used to handle changes of sign and discontinuities in the TSC84 data file, which is necessary for evaluating the logarithms of many of the entries. Since `numpy`'s `interp()` is linear, inputs are set to be the log of the relavent TSC84 parameter, and likewise the result is exponentiated. Since the function `extrap()` is written specifically for linear interpolation of a logarithmic function, the exact values from the TSC84 grid can be passed as arguments, and likewise the return is itself already re-exponentiated. This interpolation and extrapolation results in a different core density profile than was produced in the 2D codes. This difference is explained further in Section 2.4.

With the numerical parameters calculated at each point in the simulation grid, the core density profile is then calculated in the same manner as was done in the 2D-IDL code. Inside of $x < \tau^2$ the "inner solution" is calculated according to TSC84 and relies on a function `cubesolve()` which is written to exactly match its implementation in the IDL code. Outside of the inner solution region, the density is calculated using the equation from the caption of Fig. 3 in TSC84.

With a core density profile generated, it is then truncated to only fill radii between `renv_in` and `renv_out`. Re-normalization to match the total core mass is then performed in much the same way as for the accretion disk. The core and disk are then merged by making a combined density array where each cell takes whichever density is higher between the core and disk (this means that the density for $r <$ `rdisk_out` is exactly that of the disk, and $s >$ `rdisk_out` is exactly that of the core). This density profile is then scaled down by a dust to gas ratio of 1/100.

2.3.5. *The stellar source*

The final component generated for use in the radiative transfer simulations is a source stellar spectrum for the central protostar. In order to avoid errors in `radmc3d` when this spectrum has zero intensity at any wavelength, this spectrum

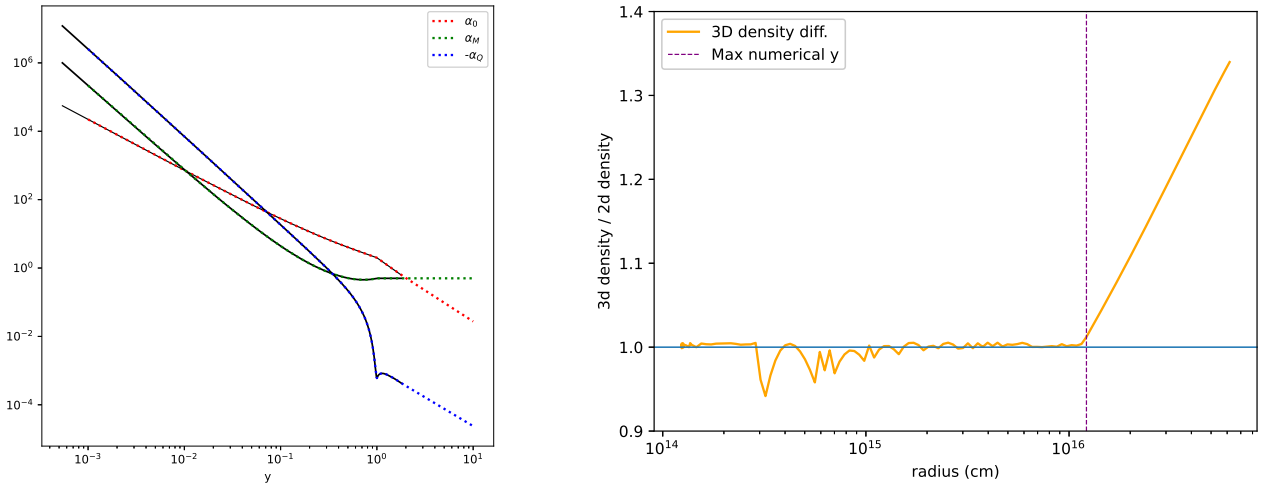

Figure 2. Results from the new interpolation approach and extrapolation algorithm. On the left: tabulated values of the α parameter from TSC84 are shown in black, with a selection of interpolated and extrapolated values plotted as colored, dotted lines. On the right: on the exact same physical grid, the 2D and 3D codes produce different density profiles. The change from interpolation to extrapolation is marked with a vertical purple line corresponding to the maximum y value from the TSC84 data file. Inside of this line, the 3D code can return lower densities corresponding to more precise slightly larger values of y and the corresponding numerical parameters. In the extrapolation regime, a more accurate determination of [one of the α parameters] produces relatively higher densities.

is manually calculated using `astropy`'s blackbody model.[3] A model object is made corresponding to the blackbody temperature at a given model timestep, and then this model used to operate on the wavelength grid returning an array of intensities. Finally, any intensity of less than 10^{-300} erg cm^{-2} s^{-1} Hz^{-1} is reset to this minimum value. This stellar spectrum is then included with the `stars.inp` file for use in `radmc3d`.

One note which is important for handling `astropy`'s model objects is that `astropy` prefers to operate in dimension*ful* quantities. As such, it must be specified that the the wavelength grid is being provided in microns (by multiplying it by a microns unit object), and then later the units of intensity must be stripped from the output spectrum so it can be handled by `numpy`.

2.4. *Differences between the 2d and the 3d models*

One of the key differences between the 2D-IDL code and the 3D-Python models lies in the creation of the core density profile. The 3D code is now built to interpolate between the entries in the TSC84 data file (`grid.plt34.mod`), whereas previously only numerical results at the nearest tabulated y value which is less than that corresponding to the actual simulation's grid point were used for calculating the density profile. The other significant improvement occurs with the extrapolation for y values exceeding the range of the TSC84 table. In the 2D code, an archaic extrapolation scheme was inherited from even older code, and produces a different values of [one of the α parameters] across the extrapolation regime, leading to a different resulting density profile. Comparisons showing the interpolation and extrapolation and resulting effects are plotted in Figure 2.

2.5. *Things which are not different between the 2d and the 3d models*

In some cases, the 2D-IDL implementation of these physical models borrows heavily from much older Fortran code, and acts in a manner which is now needlessly roundabout. In other cases, mild stylistic differences in expressing functions can make it seem as though something is was changed when moving from the 2D-IDL to 3D-Python codes. This subsection contains a non-exhaustive list of a few of these differences, which have been checked and confirmed to not affect the resultant physical model.

[3] https://docs.astropy.org/en/stable/api/astropy.modeling.physical_models.BlackBody.html

Expression of the disk scale height: An accretion disk adhering to Equation 11 of M. M. Dunham & E. I. Vorobyov (2012) with scale height constants of $H_0 = 10$ AU and $s_0 = 100$ AU (as is directly coded in the 3D scripts) can be equivalently expressed with $H_0 = 0.0316228$ AU and $s_0 = 1$ AU (as was done in the 2D code). Since

$$H_s = H_0 \left(\frac{s}{s_0} \right)^{\beta} \tag{1}$$

then either pair of constants yield the same disk scale height. For example, at 1 AU,

$$H_s(1 \text{ AU}) = 10 \left(\frac{1}{100} \right)^{1.25} = 0.0316228 \left(\frac{1}{1} \right)^{1.25} = 0.0318228 \text{ AU}. \tag{2}$$

3. RUNNING THE RADIATIVE TRANSFER SIMULATIONS

Actually computing observable signatures based on the above models requires radiative transfer simulations, which are preformed by `radmc3d` (C. P. Dullemond et al. 2012). For detailed explanations of what `radmc3d` is doing, please see its own documentation. This section instead serves to explain the choices which were made in configuring `radmc3d` to run these simulations, how they are elected in the code, and any physical explanations for them.

Running `radmc3d` is done in two steps: first, a Monte Carlo radiative transfer simulation computes dust temperatures in all grid cells; then, photons are raytraced through the grid to the observer in order to generate spectra. The steps are performed in the shell via `radmc3d` commands `mctherm` and `sed`, respectively. Each of these commands then allows for additional arguments enabling or disabling certain features of `radmc3d`. The base-line application of `radmc3d` for the 3D-Python models utilized the following flags, which are also explained in the following sections.

The thermal radiative transfer is run with a command of the form:

```
radmc3d mctherm setthreads N countwrite N
```

where `setthreads` specifies number of threads used for parallelization, and `countwrite` specifies the frequency of sign-of-life updates in units of "number of photons".

The raytracing is run with a command of the form:

```
radmc3d sed incl N useapert dpc N setthreads N
```

where `incl` specifies inclination angle measured in degrees, `useapert dpc` activates the use of a specific aperture at N kpc from the center of the simulation, and `setthreads` again specifies number of threads used for parallelization.

For the radiative transfer simulation the interstellar radiation field (ISRF) *is* included in order to account for its role in heating the dust, however the ISRF *is not* included when raytracing, since its photons would be part of a background spectrum which gets removed from equivalent physical observations.

3.1. *radmc3d controls*

In general, running `radmc3d` requires the following auxiliary files when performing a complete simulation of a model:

1. `radmc3d.inp`,
2. `wavelength_micron.inp`,
3. `dustopac.inp`,
4. `dustkappa_OH5.inp`,
5. `aperture_info.inp`, and
6. `external_source.inp`.

The purpose and use of each of these files are described in the following subsections.

3.1.1. *radmc3d.inp*

The `radmc3d.inp` file controls many of the important settings used in performing the physical radiative transfer. For flag parameters, a value of 1 enables the feature while 0 turns it off.

nphot controls the number of photons simulated in the thermal Monte Carlo. This is set to a value of 10,000,000 in order to ensure sufficient propagation through the densest regions in the inner disk, which avoids large variability

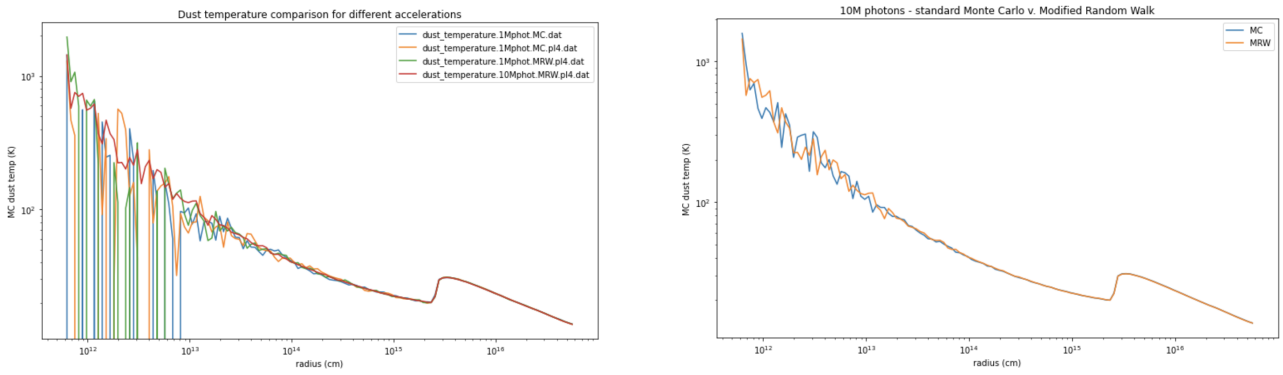

Figure 3. Dust temperatures along the midplane for model 1 timestep 90. On the left, a comparison of dust temperatures shows that with fewer than 10M photons, radiative transfer is significantly undersampled in the densest regions of the disk. On the right, a validation of the modified random walk algorithm in comparison to a true Monte Carlo simulation of the same density profile.

due to under-sampled energy transfer. Figure 3 shows an example of how this improves the precision of the simulation in the dense inner disk. For some models, even this many photons still results in undersampling, however the resultant parameters (bolometric luminosities and temperatures) are confirmed to be sufficiently accurate to justify the significantly improved computation time compared to further increasing this.

modified_random_walk is enabled, allowing `radmc3d` to utilize analytical solutions describing the diffusion of photons out of incredibly optically thick cells instead of needing to simulation copious amounts of re-emission and re-absorption events all within one cell. This *significantly* accelerates the radiative transfer simulation, by a factor of $10 - 50$, while retaining results which are equivalent to the true Monte Carlo (see Fig. 3).

scattering_mode_max is set to 1 so that both absorption and scattering of the dust will be simulated.

istar_sphere is set to 0 so that the central stellar source will be treated as a point-source of light. While this exactly matches the handling of the 2D code, a full 3D treatment of a finite-sized star would be more physical, and does not materially impact simulation complexity. Setting this flag to 1 would allow for a finite sized star with a radius matching that specified in the model parameters file.

iranfreqmode is set to 1 in all example models packaged with `radmc3d`. There is, however, no documentation of what this flag does. Even reading the source code is not particularly enlightening.

3.1.2. *wavelength_micron.inp*

The wavelength grid provided to `radmc3d` is used in all staged of the simulation. It is generated by `problem_setup.py` by reading in the frequency grid used in the 2D code, converting these frequencies to wavelengths, and then exporting along with a required declaration that there are 100 wavelengths in the grid.

When this wavelength grid is generated in Python, the order of the grid is set such that a given entry will match to its corresponding frequency from the 2D code, e.g.:

$$\texttt{wavlengths[n]} = c/\texttt{frequencies[n]}. \tag{3}$$

This allows direct matching of any other array whose values correspond to certain wavelengths/frequencies.

3.1.3. *dustopac.inp and dustkappa_OH5.inp*

Dust opacities are specified in two parts in `radmc3d`. Firstly, the file `dustopac.inp` is used to point towards each relevant dust species opacity file. Since these simulations are preformed using only OH5 dust, this file points only once, towards `dustkappa_OH5.inp`.

The dust scattering and absorption opacities are specified in `dustkappa_OH5.inp`, which requires a format declaration set to "2" (meaning both absorption and scattering are to be used); the nuber of wavelength points in the wavelength grid; and three columns reading as

$$\texttt{wavelength}\ \ \kappa_{abs}\ \ \kappa_{scat}. \tag{4}$$

This file is produced in `problem_setup.py` by re-formatting the equivalent file, `dustopac_1.inp`, from the 2D code.

3.1.4. *aperture_info.inp*

The aperture information control file sets the angular opening of the observer's aperture used when raytracing, with a size provided in arcseconds at each wavelength in the wavelength grid. This aperture is turned on when raytracing by including the flags `use apert dpc` *`distance`* in the `radmc3d sed` call.

In M. M. Dunham & E. I. Vorobyov (2012), it is only stated that a "sufficiently large" aperture is chosen for wavelengths greater than 100 μm. In the 2D-IDL code, this was implemented as an opening of 50' at these wavelengths for all models, however at the specified distance of 140 kpc this aperture corresponds to a radius much, much greater than several of the models' largest outer radii. As of right now, `problem_setup.py` generates an aperture control file which matches the 2D-IDL code, however this can be toggled by swapping the line which defines the list of four aperture sizes in the variable `aps` for one which lets the final aperture size be set by variable `vangle_arcsec`. This definition would set the largest aperture size to be one which shows, at a distance of 140 kpc, radii only up to 10% greater than the outermost simulated radius.

3.1.5. *external_source.inp*

The ISRF is generated by `problem_setup.py` by reading in an equivalent control file from the 2D code, `external_meanint.inp`. Since the wavelength grid ordering matches the original frequencies ordering, and units of erg cm^{-2} s^{-1} Hz^{-1} steradian^{-1} are consistent, this ISRF intensity is simply re-written into the `external_source.inp` file, in the format specified by `radmc3d` (single column, all wavelengths then all intensities).

3.2. *Batch scripting the radiative transfer code*

Unfortunately, `radmc3d` is quite rigid in the naming conventions it expects for input files. This means that the easiest way to run a full model in bulk is by establishing a temporary directory, (e.g., `/RUNDIR/`) in which just one timestep's files will be handled while `radmc3d` runs. At the beginning of a run, auxilliary files are copied into this directory, then for each timestep relavent inputs are copied in, results are copied out, and then the process repeats for the next timestep. This flow is also explained as the following steps:

0. The auxillairy files are copied to RUNDIR
1. stars.NNN.inp, amr_grid.NNN.inp, and dust_density.NNN.inp are copied to RUNDIR without the model number .NNN label.
2. Thermal radiative transfer is computed, and the temperatures file is copied to /OUTPUT/
3. The ISRF photons (external_source.inp) are disabled (renamed to something that radmc3d won't find)
4. Spectra for all inclinations are generated and copied to /OUTPUT/ with modelnumber and inclination labels
5. The ISRF photons are turned back on (a new external_source.inp is copied over from AUX)
6. Step up in model number and repeat beginning with step 1.

For the sake of convenience, two scripts are included within the repository (in `/rad-transfer/`) which will automatically generate these shell scripts when provided with a model number, model parameters file, and, optionally, information for partitioning the run into subsets. This latter option (only included in `runscript.ipynb` at the moment) is relevant because often the biggest bottleneck when running `radmc3d` are read/write steps, and running multiple timesteps at once (as distinct processes) is often faster than one, more highly-accelerated process. Generating scripts with four partitions, for example, will return `runmodNN-A.sh`, `runmodNN-B.sh`, `runmodNN-C.sh`, and `runmodNN-D.sh`, which should be run from distinct temporary directories (`/RUNDIR-A/` is the syntax expected, but this can be changed by modifying the Python code which generates the shell scripts).

4. ANALYSIS OF OUTPUTS

A complete simulation of a model will result in a collection of dust temperature grids and spectra for every timestep. The repository of 3D-Python code includes scripts for analyzing the output spectra as well as reading and plotting dust temperatures. These scripts are explained in the following subsections.

4.1. *Analysis of Spectra*

The most important product to extract from the outputs of `radmc3d` are bolometric luminosities and temperature for each timestep and inclination. The 3D-Python code repository contains a script, `calc_lbol_tbol.py`, for calculating these indicators for every timestep in a model and returning two tables with this information. Note that this script relies on functions included in `radmc_utils.py`, which is distributed as part of the `/make_files/` directory. It is likely easiest to just move a copy of this script into the `/analysis/` folder as well. The `calc_lbol_tbol.py` script itself can be run from within `/analysis/` and only requires setting the variable `simdir` to point towards the correct base-level directory containing model run directories and a model parameters folder.

Providing the model number will then allow bolometric temperatures and luminosities to be calculated using the functions `bol_luminosity()` and `bol_temperature()` which take as arguments an array of frequencies and a corresponding array of intensities. The functions each return a single value in units of $L_\odot$ and K respectively. The full luminosities and temperatures are exported to `lbols_modelNN.tab` and `tbols_modelNN.tab` (currently set to export to a sub-folder `/analysis/indicators/`).

I have also added an example Jupyter notebook showing the creation of 2D-3D indicators comparison plots.

4.2. *Reading the `radmc3d` Inputs and Outputs*

The `radmc_utils.py` scripts includes a few functions for reading in files which have been created for or by `radmc3d`. Equivalent scripts are also packaged with the Python distribution of `radmc3d` , however I find them to be a bit more restrictive since they make the same rigid assumptions about naming conventions that the radiative transfer code itself also does.

`load_amr_grid(file, nr, nt)` takes the file name (and path) as well as number of radial and angular grid points as arguments, and returns a tuple of 2D arrays of size $(nr+1, nt+1)$ describing the edges of each cell.

`load_dustdens(file, nr, nt)` takes the file name (and path) as well as number of radial and angular grid points as arguments, and returns a 2D array of size (nr, nt) describing the density in each cell in units of g cm^{-3}.

`load_dusttemps(file, nr, nt)` takes the file name (and path) as well as number of radial and angular grid points as arguments, and returns a 2D array of size (nr, nt) describing the temperature of each cell in units of K.

A short example notebook using these functions to plot dust density and temperature at a particular timestep is included with the repository.

REFERENCES

Dullemond, C. P., Juhasz, A., Pohl, A., et al. 2012, RADMC-3D: A multi-purpose radiative transfer tool,, Astrophysics Source Code Library, record ascl:1202.015 http://ascl.net/1202.015

Dunham, M. M., Evans, II, N. J., Terebey, S., Dullemond, C. P., & Young, C. H. 2010, ApJ, 710, 470, doi: 10.1088/0004-637X/710/1/470

Dunham, M. M., & Vorobyov, E. I. 2012, ApJ, 747, 52, doi: 10.1088/0004-637X/747/1/52

Terebey, S., Shu, F. H., & Cassen, P. 1984, ApJ, 286, 529, doi: 10.1086/162628

Vorobyov, E. I., & Basu, S. 2005, ApJL, 633, L137, doi: 10.1086/498303

Vorobyov, E. I., & Basu, S. 2006, ApJ, 650, 956, doi: 10.1086/507320

Vorobyov, E. I., & Basu, S. 2010, ApJ, 719, 1896, doi: 10.1088/0004-637X/719/2/1896

Young, C. H., & Evans, II, N. J. 2005, ApJ, 627, 293, doi: 10.1086/430436

```
simdir/
├── model_parameters/
│   └── model_parameters_1.tbl
└── model01/
    ├── AUX
    ├── INPUT
    ├── RUNDIR
    ├── OUTPUT
    └── RESULTS
star-formation/
└── make_files/
    ├── problem_setup.py
    ├── radmc_utils.py
    ├── dustopac_1.inp
    ├── external_meanint.inp
    ├── frequency.inp
    └── grid.plt34.mod
```

Figure A1. Directory structure used Colin's implementation of the 3D code.

APPENDIX

A. COLIN'S DIRECTORY STRUCTURE

As described above, absolute path definitions are used wherever possible. Ideally, this means that the code should be quite flexible with creating and storing input and output data. Still, I will attach (Fig. A1) a rough outline of the directory structure that was used by myself in the event that this is helpful for deciphering pointings or rebuilding a different structure.

The directory structure of `star-formation` matches the layout of the https://github.com/colinbourque/star-formation repository, and can exist anywhere so long as `simdir` points to the correct location.